Projet Informatique

Info 2A — programmation objet/Année universitaire 2023-2024

Bastien Duperron et Badr Habachi 29/04/2024

Pour commencer ce rapport du développement d'une application en Java, dans le cadre de notre projet de

Fin d'année de Licence (matière Info2a).

Ce projet nous a permis de mettre en pratique les différents concepts de la programmation orienté objets,

Dans cette introduction, nous présenterons brièvement le contexte du projet.

Avant d'exposer le plan détaillé du rapport et de conclure sur les enseignements tirés de cette expérience.

Le concept du projet est un jeu avec un plateau ou le l'utilisateur (joueur) peut se déplacer en lançant des

Grenades et peut interagir avec les Objets disponible dans les salles et son but est de trouver la sortie.

Nous verrons dans un premier temps les Description des Fonctionnalités, ensuite un diagramme des classes

Puis un jeu d'essais commentés, et pour finir un descriptif des classes.

Projet Informatique

Info 2A — programmation objet/Année universitaire 2023-2024

I) Descriptions des Fonctionnalisés

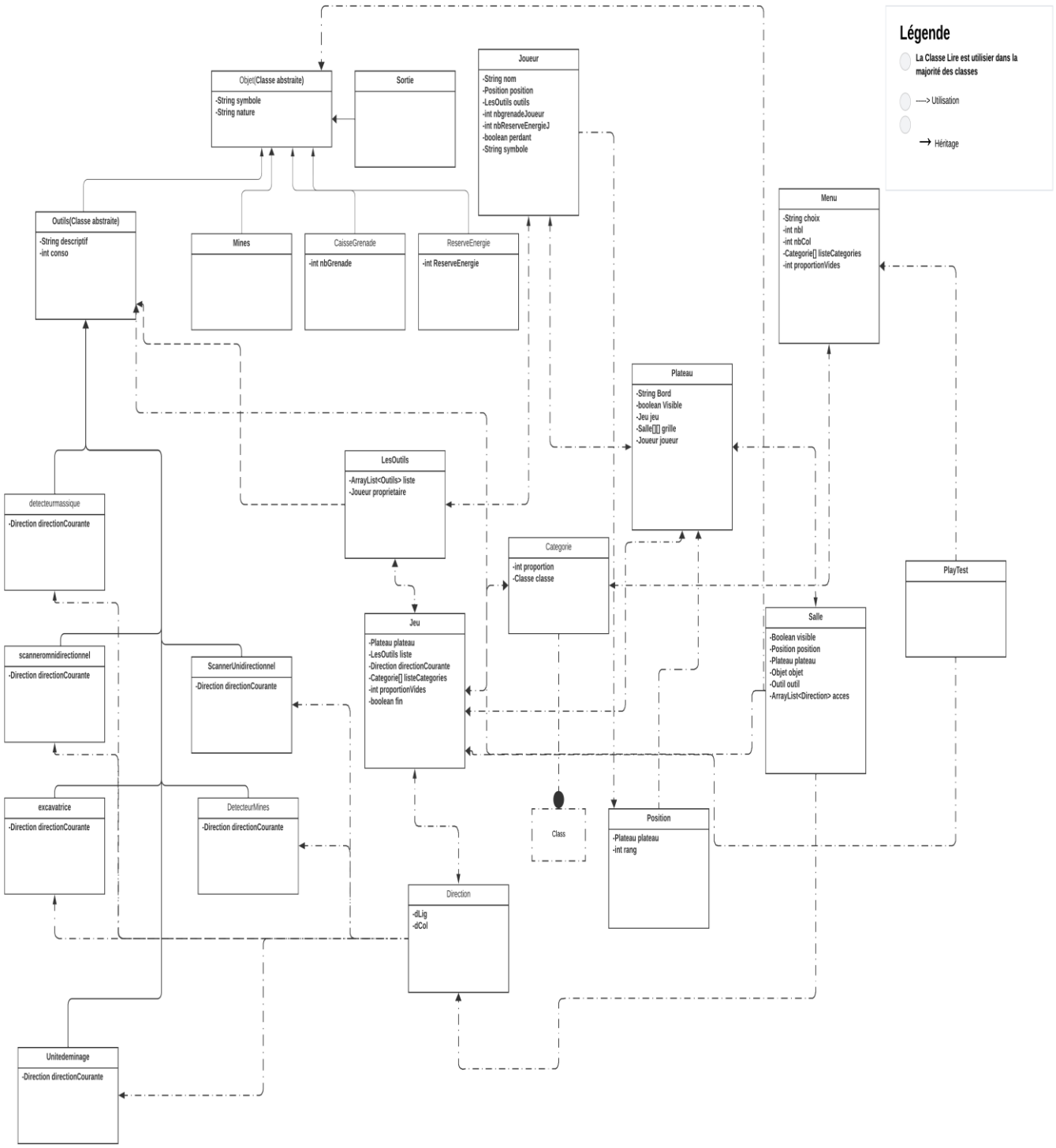
Dans cette partie de du projet, nous avons utilisé les classes de base qui nous avait été fournies par le professeur de CM.

Nous avons donc ajouté des classes supplémentaires telles que « PlayTest », excavatrice, détecteurmassique, et les classes en rapport avec les outils qui sont mentionnés dans le sujet du projet.

Nous avons donc effectué les héritages nécessaires avec les différents outils/Objets. Et les méthodes associées avec ceci telles qu'utilisation et activation qui sont dans outils des méthodes abstracts, dans chaque outil différent elles ont donc des fonctionnalités associées pour chaque d'entre elles.

De plus les fonctionnalités demandées sont implémentées dans le code, telles que la réserve d'énergie, le déplacement du joueur dans le plateau, et les interactions avec les objets et les utilisations avec les outils.

II) Diagramme de Classes



*Si affichage flou lien vers un PDF de cette photo (https://filesender.renater.fr/?s=download&token=12bf3257-090a-4c7c-8f37-51b91d455e62).

III) Jeux d'essais commentés

Ici-bas on retrouve un exemple de partie de jeu, pour illustrer le fonctionnement du programme dans sa globalité :

Le but du jeu est simple, il faut atteindre la sortie (<<), pour cela il faut utiliser des grenades pour démolir les murs autour du joueur tout en évitant de les lancer sur un mur qui cache une mine.

```
[1]Commencer
1
Quel est votre nom : JOUEUR
                §P§
choisir action:
[1]se deplacer
[2]utiler grenade
[3]utiliser outils
[4]abandonner
[5]MODE DEBUG!!!
2
Entrez une direction en combinant 'h','b','g','d' ou 'haut','bas','gauche','droite' : g
il vous reste :9 grenades
                §P§
choisir action:
[1]se deplacer
[2]utiler grenade
[3]utiliser outils
[4]abandonner
[5]MODE DEBUG!!!
```

Voici le début d'une partie, on commence par donner un pseudo à notre joueur, celui-ci on peut le reconnaître grâce au symbole § P §.

Notre joueur a 5 possibilités d'action :

-se déplacer

-utiliser une grenade

-utiliser un outil

-abandonner

-utiliser le mode de debug

Ici on commence par jeter une grenade pour créer un accès à une autre salle, dès que la grenade est jetée on se déplace immédiatement.

On remarque qu'on a un nombre de grenades définies.

Lors des déplacements, on peut nous notifier qu'un objet se trouve au sol, on peut interagir avec celui-ci, si c'est une caisse d'énergie ou une caisse de grenade, on peut recharger le joueur, si c'est un outils le joueur pourra l'équiper que s'il ne le possède pas déjà.

Ici on tombe sur un « Sun », c'est un scanner unidirectionnel.

```
                §P§
choisir action:
[1]se deplacer
[2]utiler grenade
[3]utiliser outils
[4]abandonner
[5]MODE DEBUG!!!
2
Entrez une direction en combinant 'h','b','g','d' ou 'haut','bas','gauche','droite' : b
il vous reste :9 grenades
Il y a un(e)Sun par terre.
Tapez [1] pour interagir avec ou [2]Pour ne rien faire : 1
Tapez [1] pour récupérer l'outil ou [2]Pour ne rien faire : 1
JOUEUR est déjà en possession de Sun
Vous êtes dans une salle qui contient : Sun
```

Les outils peuvent être utilisés que si le Joueur les

```
                Sun
                    cEN§P§
choisir action:
[1]se deplacer
[2]utiler grenade
[3]utiliser outils
[4]abandonner
[5]MODE DEBUG!!!
3
vous avez :
[0](Sun)Scanner unidrectionnelun détecteur massique unidirectionnel qui permet de connaître approximativement (à 10%
 + près) le nombre d'objets ? tous types confondus ? situés dans une direction donnée. Son
 utilisation nécessite 2 unités d'énergie.
vous avez :
[1](Dmi)Détecteur de minesexplore les salles autour de la salle du joueur
 (direction à droite par rapport à la position de la salle du joueur,
 + compte les mines et affiche le résultat.
Tapez le numero de l'outil que vous voulez utiliser. : 1
Nombre de mines détectées : 0
```

possédait déjà et s'il possédait assez d'énergie,

Ici on a un exemple avec le détecteur de mines, son rôle est de nous dire s'il y a une mine autour du joueur, ici aucune

```
choisir action:
[1]se deplacer
[2]utiler grenade
[3]utiliser outils
[4]abandonner
[5]MODE DEBUG!!!
2
Entrez une direction en combinant 'h','b','g','d' ou 'haut','bas','gauche','droite' : g
Vous avez trouvé la sortie!!!!
il vous reste :9 grenades
Il y a un(e)<<< par terre.
```

Le but du joueur est de trouver la sortie, celle-ci se situe dans un coin, ici dans le coin en bas à gauche (la sortie est notifiée par un <<).

IV) Descriptif des classes

Description de la classe Joueur, ses attributs et ses méthodes.

Les méthodes les plus importantes dans la classe joueur sont :

```
public void lanceGrenade(Direction d)
public void recupere()
public void avance(Direction d)
```

Commençons par la méthode lanceGrenade,

Cette méthode lanceGrenade(Direction d)` est responsable du comportement associé à l'utilisation d'une grenade par le joueur dans une direction donnée `d'.

Tout d'abord, elle récupère la position suivante dans la direction spécifiée par le joueur (`p = this.getPosition().getSuivante(d)`). Cela signifie que la grenade sera lancée dans la direction choisie par le joueur.

Ensuite, elle vérifie si le joueur a suffisamment de grenades pour lancer une grenade. Si le joueur a suffisamment de grenades, elle vérifie si la position obtenue est valide et si la position actuelle du joueur est également valide. Si la salle à la position suivante est déjà visible (ouverture de mur précédente), cela signifie que la grenade est perdue.

Si la salle à la position suivante n'est pas visible (mur fermé), alors la grenade est lancée avec succès. La salle devient visible un accès est ajouté entre la salle actuelle et la salle nouvellement ouverte(`p.getPlateau().getSalle(p).setAcces(d)`), et le joueur est déplacé dans la salle nouvellement ouverte (`this.avance(d)`).

Si la salle nouvellement ouverte contient une mine le joueur meurt ce qui met fin à la partie.

Si le joueur essaie de lancer une grenade sur un mur qui ne peut pas être brisé, un message est affiché pour l'en informer. Finalement, le nombre de grenades restantes du joueur est affiché.

Attributs de Joueur.

La classe joueur possède des Attributs de différents types tel que :

```
private String nom;
private Position position; Permet de récupérer la position du joueur et de la mettre a jour
private LesOutils outils; Cela vas être la liste des outils du joueur
private int nbgrenadeJoueur;
private int nbReserveEnergieJ;

private boolean perdant;
 private String symbole="§P§";
```

Pour continuer la méthode recupere() est responsable de l'interaction entre le joueur et un outil présent dans une salle.

La méthode commence par initialiser deux compteurs cpt et o à zéro. Pour contrôler le choix du joueur.

Ensuite, elle entre dans une boucle while qui continue tant que cpt est égal à zéro. Tant que le joueur n'a pas fait un choix valide, la boucle continue de demander à l'utilisateur de faire un choix.

À l'intérieur de la boucle while, elle demande à l'utilisateur de choisir entre récupérer l'outil (1) ou ne rien faire (2).

Si l'utilisateur choisit 1, la méthode ajoute l'outil contenu dans la salle à l'inventaire du joueur en utilisant la méthode ajoute () de la classe LesOutils. Avant d'ajouter l'outil, il est casté en tant qu'objet de type Outil.

Si l'utilisateur choisit 2, la méthode affiche un message pour informer le joueur que l'objet n'a pas été ajouté.

Si l'utilisateur entre un nombre différent de 1 ou 2, la méthode affiche un message indiquant que le numéro est invalide et redemande à l'utilisateur de faire un choix.

Une fois que le joueur a fait un choix valide (récupérer l'outil ou ne rien faire), le compteur cpt est incrémenté, ce qui permet de sortir de la boucle while et de terminer la méthode.

Ensuite la méthode avance (Direction d) permet au joueur d'avancer dans une direction donnée sur le plateau de jeu, à condition que le mouvement soit valide.

La méthode commence par récupérer la position suivante dans la direction donnée d en utilisant la méthode getSuivante() de la classe Position. Cette position est stockée dans la variable p.

Ensuite, la méthode vérifie si la position p est valide en appelant la méthode isValide() de la classe Position. Si la position n'est pas valide (en dehors des limites du plateau), un message d'erreur est affiché et la méthode se termine.

Si la position est valide, la méthode vérifie si la direction d est valide et si la salle correspondante sur le plateau est visible. Cela garantit que le joueur ne peut avancer que vers une salle qui est visible et accessible.

Si la salle est visible, la méthode met à jour la visibilité de la salle suivante à vrai en appelant setVisible(true), déplace le joueur vers cette nouvelle position en utilisant setPosition(p), et vérifie si le joueur a atteint une sortie. Si c'est le cas, un message est affiché pour informer le joueur qu'il a trouvé la sortie et le jeu se termine en appelant isFini() de la classe Jeu.

Si la salle n'est pas visible le mur est fermé ou la salle est hors des limites, un message est affiché pour informer le joueur qu'il ne peut pas sortir du plateau.

Si la position p n'est pas valide, un message d'erreur est affiché indiquant que la position est invalide.

Description de la classe Détecteur de Mines.

Les méthodes principales sont :

```
    @Override
    public void activation(Joueur j)
    @Override
    public void utilisation(Joueur j)
```

Cette méthode ***activation*** (Joueur j) est une fonctionnalité spécifique à un outil (ici, un scanner unidirectionnel) qui permet au joueur de détecter les mines dans les salles environnantes de sa position actuelle sur le plateau de jeu. Voici une explication détaillée de son fonctionnement :

La méthode commence par initialiser un compteur cptMines à zéro, qui sera utilisé pour compter le nombre de mines détectées.

Ensuite, elle récupère le plateau de jeu associé au joueur passé en paramètre, ainsi que la position actuelle du joueur.

Elle crée un tableau de directions *DIRECTIONS* qui contient toutes les directions possibles pour explorer les salles environnantes. Ces directions incluent haut, bas, gauche, droite, et les cotées haut. La méthode réitère à travers chaque direction dans le tableau DIRECTIONS en utilisant une boucle for-each. Pour chaque direction, elle calcule la position de la salle voisine en utilisant la méthode getSuivante() de la classe Position, qui prend en compte la direction donnée.

Ensuite, elle récupère la salle voisine correspondante sur le plateau en utilisant la méthode getSalle(Position) de la classe Plateau. Si la salle voisine existe (c'est-à-dire qu'elle n'est pas en dehors des limites du plateau), elle récupère son contenu. Si le contenu de la salle voisine est une instance de la classe Mine, cela signifie qu'une mine a été détectée. Dans ce cas, le compteur cptMines est incrémenté.

Après avoir exploré toutes les directions, la méthode affiche le résultat de la détection en indiquant le nombre de mines détectées.

Attributs de la classe Mine

```
private Direction directionCourante;
Permet de demander scanner dans le sens trigonométrique les salle au tour du joueur grâce au constructeur suivant new Direction("d"); dans la classe Direction.
```

La classe Mine est une classe qui hérite de la classe Outils elle contient donc des méthodes qui sont de la classe Outils.

Description de plateau.

Les méthodes principales sont :

```
 public Plateau(int nbLig, int nbCol, Jeu jeu)
public void initPlateau ()
```

Dans le constructeur de celui-ci, on demande en paramètre le nombre de colonne, le nombre de ligne et un attribut de type jeu.

Grâce au nombre de ligne et de colonne on initialise la taille de la matrice Grille et on initialise chaque salle de cette matrice grâce à la méthode setNouvelleSalle(),a cette étape on possède un plateau avec une taille et toute ses pièces vide.

Par la suite on initialise le joueur avec sa position au centre du plateau et on rend cette salle visible.

A la fin du constructeur on fait appel à initPlateau() qui va s'occuper de placer des objet dans les salles du plateau.

Dans la méthode initPlateau() on place au hasard une sortie dans un des coins du plateau, par la suite on fait appel à la méthode placeObjet() qui quant a elle place le reste des objet.

placeObjet() récupère la liste de categorie (on retrouve cet attribut dans la class Jeu)en paramètre et calcul le nombre chaque objet dans le plateau.

Après cela on place aux hasard les objets dans les salles tout en vérifiant qu'on les places bien dans des salles vides et qu'on les place pas au centre du plateau(ou débute le joueur)

Attributs de la classe Plateau

```
private static final String BORD
public static boolean VISIBLE
private Jeu jeu;
private boolean visible;
private Salle[][] grille;
private Joueur joueur;
```

Description de la classe Jeu.

Les méthodes principales sont :

Dans le constructeur de cette class on retrouve en paramètre le nombre de colonne et de ligne du plateau, la proportion de salle vide et un tableau de catégories.

Dans celui-ci on initialise l'attribut listeCategories et proportionVides et on crée une instance de Plateau avec en paramètre la longueur et la largeur et le jeu (donc this) dont l'adresse est affectée à l'attribut plateau.

A la fin du constructeur on fait appelle à joue().

Dans la méthode joue() on retrouve une boucle do while(), cette boucle permet de répéter en boucle le tour du joueur tant qu'il ne meurt pas, abandonne pas ou ne gagne pas.

Dans le tour on commence par vérifier grâce à un if() si le joueur est dans une salle avec un objet.

Après ça, grâce à un switch on lui fait choisir entre se déplacer, lancer une grenade, utiliser un outil, abandonner ou utiliser le debug mode.

Pour le déplacement le joueur est invité à entrer une direction valide (haut, bas, gauche ou droite). Ensuite, le joueur avance dans la direction spécifiée. S'il y a un objet dans la salle où le joueur se déplace, le joueur a la possibilité d'interagir avec cet objet.

Pour le lancement de grenade, de manière similaire au premier cas, le joueur est invité à entrer une direction valide. Ensuite, le joueur lance une grenade dans la direction spécifiée. S'il y a un objet dans la salle où la grenade atterrit, le joueur a la possibilité d'interagir avec cet objet

Pour utiliser un outil, Le joueur voit la liste des outils disponibles et peut choisir celui qu'il souhaite utiliser. Une fois qu'il a sélectionné un outil, il l'utilise.

Pour abandonner, La méthode isFini() est appelée pour déterminer si le jeu est terminé ou non.

Pour le mode débogage, Si le plateau est visible, il devient invisible, et vice versa.

Attributs de la classe Jeu

```
private Plateau
plateau;
private LesOutils
liste;
private Direction
directionCourante;
private Categorie[]
listeCategories;
private int
proportionVides;
private boolean
fin=false;
```